

Unit – II

Software Requirements Engineering and Analysis

1. Modeling

Modeling in software engineering is the process of creating abstract representations of a software system to understand, analyze, design, and communicate its structure and behavior before actual development. These models help in visualizing the system from different perspectives and ensure that all stakeholders have a clear understanding of the system.

1.1 Requirements Engineering

Requirements Engineering (RE) is the process of **gathering, analyzing, documenting, and managing the requirements** of a software or system to ensure it meets stakeholders' needs. It is a crucial phase in software development that helps avoid project failures due to unclear or incomplete requirements.

Requirement

A **requirement** is a **documented need, condition, or capability** that a system, software, or product must meet to fulfill stakeholder expectations. It serves as a guideline for development, ensuring that the final product works as intended.

Types of Requirements

1. **Functional Requirements** – Define specific functions or features the system must have (e.g., user authentication, data processing).
2. **Non-Functional Requirements** – Specify system qualities like performance, security, usability, and scalability.
3. **Business Requirements** – Describe high-level needs and goals from a business perspective.
4. **User Requirements** – Define what users need from the system in terms of interaction and experience.

5. **System Requirements** – Detail the technical specifications for hardware, software, and infrastructure.
6. **Performance Requirements** – Set benchmarks for speed, responsiveness, and efficiency.
7. **Security Requirements** – Outline data protection, authentication, and access control measures.
8. **Regulatory & Compliance Requirements** – Ensure adherence to legal and industry standards.

Importance of Requirement Engineering

1. **Clear Understanding:** Ensures a clear and shared understanding of project goals among stakeholders.
2. **Reduces Risks:** Identifies potential issues early, minimizing project risks and failures.
3. **Cost Efficiency:** Prevents costly changes and rework by defining requirements upfront.
4. **Improves Quality:** Ensures the final product meets user needs and expectations.
5. **Enhances Communication:** Bridges the gap between stakeholders and development teams.
6. **Guides Development:** Provides a roadmap for design, development, and testing.
7. **Ensures Compliance:** Helps meet regulatory and business standards.
8. **Facilitates Scalability:** Allows for future growth and adaptability of the system.
9. **Boosts User Satisfaction:** Delivers a product that aligns with user needs and expectations.
10. **Supports Decision-Making:** Provides a basis for prioritizing features and resources.

1.2 Requirements Engineering Tasks

1. Inception: This is the initial phase where the need for a software system is identified, and key stakeholders (clients, users, developers) are recognized.

Key Activities:

- Identifying project stakeholders (who will use or be affected by the system).
- Understanding the problem domain.

- Discussing the high-level goals and objectives of the software.
- Defining preliminary system boundaries.
- Assessing the feasibility of the project.

Example: A company wants to develop an e-learning app. In this stage, they identify teachers, students, and administrators as stakeholders and define the goal of creating an interactive learning platform.

2. Elicitation: The process of gathering requirements from stakeholders using various techniques.

Key Activities:

- Conducting interviews with users, clients, and domain experts.
- Using questionnaires and surveys to collect feedback.
- Observing current systems and workflows.
- Conducting brainstorming sessions.
- Reviewing similar existing systems.
- Performing Joint Application Development (JAD) sessions.

Example: Developers interview teachers to understand what features they need in the e-learning app, such as quizzes, live sessions, and progress tracking.

3. Elaboration: This phase refines and expands the gathered requirements to make them more detailed and structured.

Key Activities:

- Developing models (Use Case Diagrams, Data Flow Diagrams, etc.).
- Identifying functional and non-functional requirements.
- Creating detailed descriptions of system behavior.
- Establishing relationships between different requirements.

Example: After discussing with teachers, developers create detailed use case diagrams for user roles like "Student," "Teacher," and "Admin," showing how they interact with the system.

4. Negotiation

Since different stakeholders may have conflicting requirements, this phase ensures a balanced and mutually agreeable solution.

Key Activities:

- Identifying conflicting requirements.
- Prioritizing critical features based on business value and feasibility.
- Resolving disagreements through discussions and trade-offs.
- Documenting accepted requirements.

Example: Teachers want real-time video streaming, but developers say it increases server costs. They negotiate and agree to include pre-recorded video lectures as a cost-effective alternative.

5. Specification

The process of formally documenting the agreed-upon requirements in a structured manner.

Key Activities:

- Writing a Software Requirement Specification (SRS) document.
- Using structured formats (text, diagrams, models).
- Defining functional, non-functional, and business requirements.

Example: Developers create an SRS document specifying that the system should support "student login, course enrollment, and progress tracking" with detailed functionality descriptions.

6. Validation

Ensures that the documented requirements are correct, complete, and feasible before development begins.

Key Activities:

- Conducting requirement reviews with stakeholders.
- Using prototyping to validate requirements.
- Performing test case generation to check requirement clarity.

- Identifying inconsistencies, ambiguities, or missing requirements.

Example: Developers create a prototype of the e-learning app's login and course dashboard. Teachers and students test it and suggest improvements before finalizing the requirements.

7. Requirements Management

This phase deals with tracking, controlling, and updating requirements throughout the project lifecycle.

Key Activities:

- Maintaining a requirements database or repository.
- Handling changes in requirements systematically.
- Ensuring traceability (linking requirements to design and testing).
- Version control for requirements documentation.

Example: After launching the app, users request a "dark mode" feature. This change is evaluated, documented, and integrated into future updates while keeping track of modifications.

1.3 Establishing the Groundwork

Establishing the Groundwork in Software Engineering refers to the initial phase of a project where the foundation is laid for successful development. This phase involves defining the project's scope, objectives, and constraints, as well as identifying stakeholders and understanding their needs. It sets the stage for all subsequent activities and ensures that the team has a clear direction and shared understanding of what needs to be achieved.

Example Scenario:

A company wants to develop a mobile app for online food delivery. During the "Establishing the Groundwork" phase:

- The problem statement is defined: "Create an app to connect customers with local restaurants for food delivery."
- Stakeholders include customers, restaurant owners, delivery personnel, and the development team.

- Objectives are set: "Provide a user-friendly interface, real-time order tracking, and secure payment options."
- The scope is determined: "The app will handle order placement and tracking but will not manage restaurant inventory."
- Feasibility studies confirm the project is viable within the budget and timeline.
- A vision document is created to guide the team and stakeholders.

1.4 Identifying Stakeholders

Identifying stakeholders in software engineering is the process of recognizing all individuals, groups, or entities that have an interest in or are affected by the software development project. Stakeholders play a crucial role in defining requirements, setting expectations, and influencing project decisions.

Types of Stakeholders in Software Engineering

1. **Primary Stakeholders** (Directly affected by the system)
 - **Clients** – The organization or individuals funding the project.
 - **End-Users** – The people who will use the software (customers, employees, students, etc.).
 - **Project Managers** – Oversee project timelines, budgets, and deliverables.
2. **Secondary Stakeholders** (Indirectly affected by the system)
 - **Software Developers** – Design, implement, and test the software.
 - **UI/UX Designers** – Ensure the system is user-friendly.
 - **Quality Assurance (QA) Testers** – Validate that the software meets requirements.
 - **System Administrators** – Manage deployment, maintenance, and security.
3. **Regulatory & Compliance Stakeholders**
 - **Legal and Compliance Teams** – Ensure adherence to laws, security standards, and industry regulations.
 - **Government or Industry Regulators** – Provide guidelines for software development in regulated industries (e.g., healthcare, finance).

4. External Stakeholders

- **Suppliers & Vendors** – Provide third-party services, tools, or software integrations.
- **Investors** – Fund the project and expect business returns.

How to Identify Stakeholders?

- **Brainstorming Sessions** – Discuss potential stakeholders with the project team.
- **Interviews & Surveys** – Gather insights from business owners, end-users, and domain experts.
- **Stakeholder Analysis Matrix** – Classify stakeholders based on their level of influence and interest.
- **Reviewing Documentation** – Analyze contracts, business plans, or previous project reports.

1.5 Recognizing Multiple Viewpoints

Recognizing Multiple Viewpoints in Software Engineering refers to the process of understanding and addressing the diverse perspectives, needs, and expectations of different stakeholders involved in a project. Since stakeholders often have varying roles, interests, and priorities, their viewpoints on the system's requirements and functionality may differ. Recognizing and reconciling these multiple viewpoints is essential for creating a system that satisfies all stakeholders and avoids conflicts.

Steps to Recognize and Manage Multiple Viewpoints:

1. Identify Stakeholders:

- List all individuals or groups with an interest in the project (e.g., end users, clients, developers, regulators).

2. Gather Viewpoints:

- Use techniques like interviews, surveys, workshops, or focus groups to collect stakeholder perspectives.
- Example: Ask users about their pain points and ask developers about technical constraints.

3. Analyze Viewpoints:

- Compare and contrast the different viewpoints to identify commonalities and conflicts.
- Example: Users may want a feature that developers consider technically challenging.

4. Categorize Viewpoints:

- Group similar viewpoints to simplify analysis and prioritization.
- Example: Group all usability-related requirements from different user roles.

5. Reconcile Conflicts:

- Facilitate discussions to resolve conflicts and find compromises.
- Example: Balance user demands for features with development constraints by prioritizing must-have features.

6. Document Viewpoints:

- Record all viewpoints and decisions in a structured format (e.g., requirements document).
- Ensure transparency and traceability.

7. Validate Viewpoints:

- Confirm with stakeholders that their viewpoints have been accurately understood and addressed.
- Use prototypes or mockups to demonstrate how their needs will be met.

1.6 Working Toward Collaboration

Working toward collaboration in software engineering refers to the process of fostering teamwork and effective communication among all stakeholders—developers, designers, testers, clients, and end-users—to ensure the successful development of a software project. Since software development is a complex and iterative process, collaboration helps align goals, resolve conflicts, and improve efficiency.

Key Aspects of Effective Collaboration**1. Clear Communication**

- Use structured documentation and meetings to align expectations.
- Maintain transparency in decisions and project progress.

2. Cross-Functional Teamwork

- Developers, designers, testers, and business analysts should work together rather than in isolation.
- Encourage knowledge sharing between different roles.

3. Agile Methodology

- Use Agile practices like **Scrum** or **Kanban** to promote iterative development and continuous collaboration.
- Conduct **daily stand-up meetings** to discuss progress and roadblocks.

4. Version Control & Code Collaboration

- Use **Git**, **GitHub**, or **GitLab** for managing code versions and enabling multiple developers to work on the same project.
- Conduct **code reviews** and **pair programming** to maintain code quality.

5. Requirement Gathering with Stakeholders

- Involve clients, users, and business analysts in requirement discussions.
- Use techniques like **brainstorming**, **interviews**, and **prototyping** to align needs.

6. Feedback & Iteration

- Encourage frequent feedback loops through testing, prototyping, and usability evaluations.
- Implement **continuous integration/continuous deployment (CI/CD)** to streamline updates.

7. Conflict Resolution & Team Bonding

- Address conflicts constructively by encouraging open discussions.
- Build a positive work culture with mutual respect and accountability.

1.7 Asking the First Questions

"Asking the First Questions" in software engineering refers to the initial phase of a project where fundamental questions are posed to understand the problem, define requirements, and set the direction for development. These questions help in gathering critical information about the project's scope, objectives, constraints, and stakeholders.

Key Questions to Ask in the Initial Phase

1. Understanding the Problem Domain

- What problem is the software intended to solve?
- Who are the primary users, and what are their needs?
- Are there any existing solutions, and what are their shortcomings?

2. Identifying Stakeholders

- Who are the key stakeholders (clients, users, developers, regulators, etc.)?
- What are their expectations from the software?
- How will different stakeholders interact with the system?

3. Defining Requirements

- What are the core functional requirements (features, operations, data handling)?
- What are the non-functional requirements (performance, security, scalability)?
- Are there any legal, ethical, or regulatory constraints?

4. Technical Considerations

- What technology stack (programming languages, frameworks, databases) will be used?
- What are the system integration requirements?
- Are there any infrastructure or hosting considerations?

5. Project Feasibility & Constraints

- What is the estimated budget and timeline for development?
- What resources (team, tools, infrastructure) are available?
- What are the potential risks and mitigation strategies?

6. Expected Outcomes & Success Criteria

- What are the key performance indicators (KPIs) for project success?
- How will the software be tested and validated?
- How will updates and maintenance be handled post-deployment?

1.8 Eliciting Requirements

Eliciting requirements is the process of gathering, understanding, and documenting the needs and expectations of stakeholders for a software system. It is a crucial phase in **Requirements Engineering**, as it ensures that the final product aligns with user needs, business goals, and technical constraints.

Key Techniques for Requirements Elicitation

1. Interviews

- One-on-one discussions with stakeholders to understand their expectations.
- Can be structured (predefined questions) or unstructured (open discussions).

2. Surveys & Questionnaires

- Used to collect feedback from a large number of stakeholders efficiently.
- Helps in understanding preferences and common pain points.

3. Observation

- Analysts observe how users interact with existing systems to identify issues and improvements.
- Useful for understanding implicit requirements that users may not express directly.

4. Brainstorming Sessions

- Involves stakeholders discussing ideas, potential features, and system improvements.
- Encourages innovative thinking and collaborative decision-making.

5. Prototyping

- Creating a preliminary model of the software to gather feedback before full development.

- Helps stakeholders visualize and refine requirements.

6. Use Cases & Scenarios

- Defines specific situations in which users will interact with the system.
- Helps in understanding functional requirements and user workflows.

7. Document Analysis

- Reviewing existing documentation, business reports, and previous system requirements.
- Helps in identifying constraints, compliance requirements, and dependencies.

8. Workshops & Joint Application Development (JAD)

- Interactive sessions involving developers, designers, and users to collaboratively define requirements.
- Helps in resolving conflicts and aligning expectations.

General Process Models of Requirements Elicitation

1. Traditional Process Model: This model follows a **linear sequence** for gathering requirements.

Steps:

1. **Identify Stakeholders** – Determine who will provide requirements (users, clients, managers).
2. **Collect Requirements** – Use interviews, surveys, and documents to gather information.
3. **Analyze Requirements** – Identify feasibility, conflicts, and missing information.
4. **Document Requirements** – Create a Software Requirement Specification (SRS).
5. **Validate Requirements** – Ensure correctness and completeness with stakeholders.

Best for: Well-defined projects with clear needs.

Limitation: Not flexible for changing requirements.

2. Prototyping Model: This model involves building a **working prototype** to understand user needs better.

Steps:

1. **Gather Initial Requirements** – Collect preliminary needs.
2. **Develop Prototype** – Build a basic version of the system.
3. **Get User Feedback** – Stakeholders interact with the prototype.
4. **Refine Requirements** – Modify based on feedback.
5. **Finalize Requirements** – Once satisfied, document them for development.

Best for: Complex or unclear projects needing user interaction.

Limitation: Time-consuming and may lead to unrealistic expectations.

3. Joint Application Development (JAD) Model: JAD involves **collaborative workshops** with stakeholders and developers to define requirements.

Steps:

1. **Conduct JAD Sessions** – Bring users, developers, and analysts together.
2. **Brainstorm & Discuss** – Identify needs, expectations, and constraints.
3. **Create Models & Diagrams** – Represent system processes visually.
4. **Review & Finalize** – Stakeholders agree on requirements.

Best for: Large enterprise systems needing agreement from multiple stakeholders.

Limitation: Requires extensive time and scheduling efforts.

4. Cognitive Model: This model focuses on understanding how users **think and behave** when interacting with a system.

Steps:

1. **Observe Users** – Study how they use existing systems.
2. **Analyze Thought Process** – Identify pain points and expectations.
3. **Define Requirements** – Document insights into formal requirements.

Best for: UX/UI-focused projects where user behavior matters.

Limitation: May not cover all technical requirements.

5. Contextual Inquiry Model

This model involves gathering requirements **in the real environment** where the system will be used.

Steps:

1. **Visit Users' Workplaces** – Observe tasks and workflows.
2. **Conduct Interviews** – Ask questions while they perform tasks.
3. **Analyze & Extract Requirements** – Identify needs based on real-world data.

Best for: Systems requiring deep domain knowledge (e.g., medical or industrial software).

Limitation: Time-intensive and may not be scalable for large projects.

6. Agile-Based Elicitation Model

This model follows an **iterative approach**, gathering requirements **continuously** through user feedback.

Steps:

1. **Identify Initial Requirements** – Basic features needed.
2. **Develop Small Iterations** – Deliver working software frequently.
3. **Get Feedback** – Users test and provide input.
4. **Refine & Adjust** – Modify requirements as per feedback.

Best for: Projects with evolving requirements (e.g., startups and dynamic markets).

Limitation: May lead to scope creep if not controlled.

1.9 Collaborative Requirements Gathering

Collaborative Requirements Gathering (CRG) is a structured, team-oriented approach in software engineering where developers, stakeholders, and customers work together to define, refine, and document the requirements of a software system. This ensures that all perspectives are considered, leading to a well-balanced and clearly defined set of software requirements.

One of the most widely used techniques for CRG is the **Facility Application Specification Technique (FAST)**, where a joint team of developers and customers collaborates to identify the problem, propose solutions, negotiate approaches, and prepare an initial specification.

FAST Approach for Collaborative Requirements Gathering

FAST involves a **structured meeting** where stakeholders and developers collectively discuss and refine the requirements. The process follows these guidelines:

1. **Neutral Meeting Site:** The meeting should be conducted at a neutral location where both customers and developers can communicate openly.
2. **Preparation Rules:** Clear guidelines should be set for participation, including defining roles, objectives, and expected contributions.
3. **Agenda Planning:** The meeting agenda should cover all key points while allowing room for innovative ideas and discussions.
4. **Facilitator Role:** A facilitator (who can be a customer, developer, or an outsider) ensures smooth discussions, manages conflicts, and maintains focus.
5. **Definition Mechanism:** Work tools like worksheets, flip charts, electronic boards, or virtual forums are used to document discussions and ideas.
6. **Goals of FAST**
 - Identify the problem
 - Define solution elements
 - Negotiate different approaches
 - Prepare an initial specification for the software requirements

Example: FAST for a Video Game System (Gamefun)

1) List of Objects

- **Display** – The game interface.
- **Menu** – Start, exit, and game settings.
- **Sound** – Different sound options.
- **Event** – Game progression (moving between levels).

2) List of Services

- **Setting Sounds** – Adjusting game audio.
- **GUI Customization** – Changing color themes.
- **Help & Instructions** – Providing user guidance.
- **Scoreboard** – Displaying player scores.

3) List of Constraints

- **User-friendly interface** – Should be easy to navigate.
- **High speed** – Smooth performance without lag.
- **Low memory usage** – Should not consume excessive system resources.
- **Cost efficiency** – Should be affordable.

1.10 Usage Scenarios

A *usage scenario* is a specific sequence of interactions between a user and a software system, describing how the system is expected to behave in a given situation. It helps in understanding system requirements, designing user interfaces, and testing system functionality.

Key Aspects of Usage Scenarios

1. **User-Centered** – Focuses on how users interact with the system.
2. **Step-by-Step Interaction** – Describes a logical flow of actions.
3. **Real-World Context** – Represents practical use cases.
4. **Goal-Oriented** – Helps achieve specific user objectives.
5. **Can Be Detailed or High-Level** – Ranges from broad descriptions to precise step-by-step actions.

Types of Usage Scenarios

1. Normal Scenario (Happy Path)

- Represents the expected and error-free interaction.

- **Example:** A user logs into an e-commerce website, searches for a product, adds it to the cart, and completes the purchase.

2. Alternative Scenario

- Describes variations in the normal flow due to different user choices or system conditions.
- Example: A user selects a different payment method or applies a discount code.

3. Exception Scenario

- Describes what happens when an error or unexpected event occurs.
- Example: A user enters incorrect login credentials, leading to an error message.

Example of a Usage Scenario

Scenario: User Registration in a Web Application

1. The user opens the website.
2. Clicks on the "Sign Up" button.
3. Fills in personal details (name, email, password).
4. Clicks "Register."
5. The system validates the input and sends a confirmation email.
6. The user clicks the verification link in the email.
7. The system activates the account, allowing the user to log in.

1.11 Elicitation Work Products

Elicitation work products are the **artifacts or documents** created during the **requirements elicitation** phase in software engineering. These work products help in capturing, analyzing, and refining system requirements to ensure that the final software meets user needs.

Key Elicitation Work Products

1. Stakeholder Needs List

- A document listing all key **stakeholders** and their expectations from the system.

2. Use Cases & Usage Scenarios

- **Use Cases:** Describe how users interact with the system to achieve specific goals.
- **Usage Scenarios:** Step-by-step interaction flows between users and the system.

3. User Stories (Agile Approach)

- Short, simple descriptions of system features from an **end-user perspective** (e.g., "As a customer, I want to reset my password so that I can access my account again").

4. Requirements Specification Documents

- **Software Requirement Specification (SRS):** A formal document detailing functional and non-functional requirements.
- **Business Requirement Document (BRD):** Captures high-level business needs.

5. Prototypes & Wireframes

- **Prototypes:** Interactive mockups of the system to visualize functionality.
- **Wireframes:** Static blueprints of the system's UI.

6. Process Models & Diagrams

- **Flowcharts:** Show step-by-step processes.
- **Data Flow Diagrams (DFDs):** Illustrate data movement in the system.
- **Entity-Relationship Diagrams (ERD):** Represent database relationships.

7. Meeting Notes & Interview Summaries

- Documentation of **stakeholder meetings, interviews, and workshops** to ensure key points are recorded.

8. Risk Analysis Reports

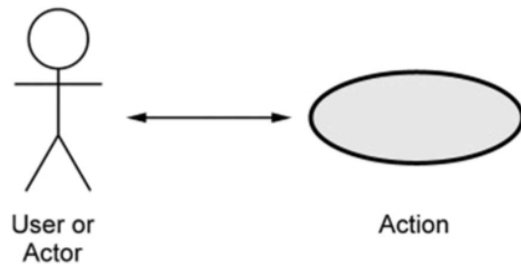
- Identifies **potential risks** in system development and suggests mitigation strategies.

9. Feasibility Study Report

- Evaluates the project's **technical, economic, and operational feasibility** before development begins.

1.12 Developing Use Cases

A *use case* is a fundamental modeling unit in software engineering that distinctly represents a system's functionality. It depicts the software/system from an **end-user's perspective**, ensuring that all necessary interactions between the user and the system are well-defined.



Steps in Developing Use Cases

1. Identifying Actors

- **Actors** are external entities that use or interact with the system to achieve a goal.
- Each actor represents a specific **role** rather than an individual person.
- Actors can be:
 - **Primary Actors** – Directly interact with the system to perform tasks.
 - **Secondary Actors** – Support the system indirectly to help primary actors achieve their goals.

Example: In an online banking system,

- A **Customer** (primary actor) performs transactions.
- A **Bank Server** (secondary actor) processes requests in the background.

2. Iterative Requirement Elicitation

- Identifying **all actors** in a single step is not always possible.
- Initially, **primary actors** are identified.
- In later iterations, **secondary actors** are included.

✦ *Example:* In an e-commerce platform,

- **First Iteration:** Identifying customers who browse and purchase products.
- **Later Iterations:** Adding roles like **admin (who manages inventory)** and **payment gateway (which processes transactions)**.

3. Developing the Use Case

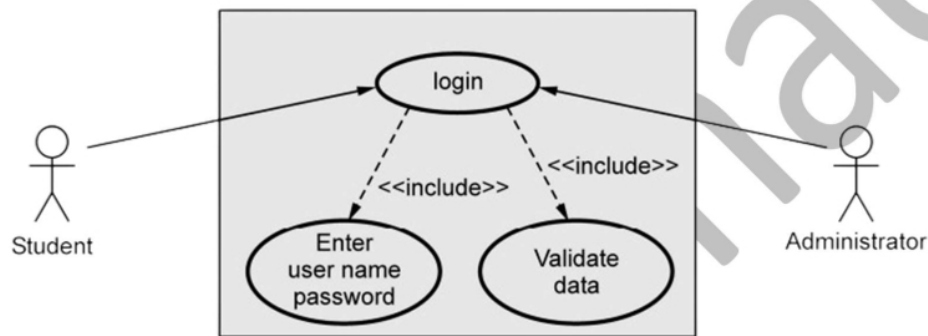
To define a use case, **Jacobson's Questions** should be answered:

Question	Example for an Online Banking System
1. Goals of the system?	Provide banking services like balance checking, fund transfers, bill payments.
2. Who are the primary and secondary actors?	Primary: Customer, Secondary: Bank database, Payment Gateway.
3. Preconditions before system development?	Bank database should be operational, users must be registered.
4. Main functions performed by the actor?	Login, check balance, transfer funds, pay bills.
5. Exceptions to be handled?	Insufficient funds, incorrect login details, transaction failures
6. Changes in actor behavior?	A user may update account details, switch accounts.
7. What system info does the actor produce?	Transaction logs, account updates.
8. What info does the actor receive?	Account balance, transaction confirmation.
9. Will the actor inform external systems?	Yes, alerts may be sent to mobile/email.

Example: Write a use case for 'login' with a template and diagram.

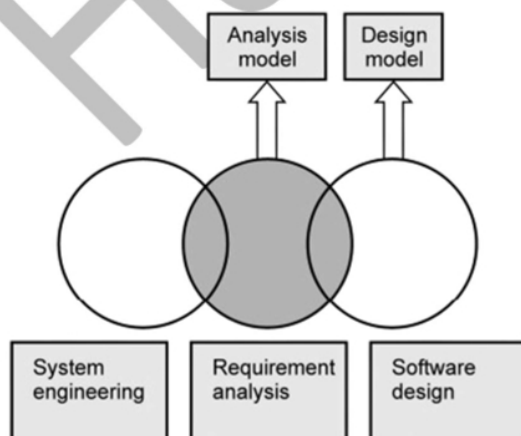
Use case template for login	
Use Case	Course Registration System
Primary Actor	Student
Goal in Context	To monitor all the functionalities required to register for the course
Preconditions	None
Scenario / Basic Flow	1. The system requests student to enter his name and password. 2. The student enters his/her name and password.

	3. The system validates the entered name and password and logs the student into the system.
Exceptions	1. The control panel is not ready. 2. User name is invalid. 3. Password is incorrect.
Priority	Essential and must be implemented in course registration system.
Postcondition	If user logs in successfully then course information must be displayed to him / her.
Secondary Actor	Administrator.



1.13 Building the Requirements Model

Building a **Requirements Model** involves structuring and representing the collected requirements in a way that helps stakeholders understand, analyze, and validate them before moving to system design and implementation.



Key Steps in Building a Requirements Model

1. Eliciting Requirements

- Gather customer needs using interviews, surveys, brainstorming, and workshops.
- Identify functional and non-functional requirements.

Example:

- A user wants **secure login** with two-factor authentication.
- The system must **support 1000 concurrent users**.

2. Categorizing Requirements

- **Functional Requirements:** Define system behavior (e.g., "User must be able to reset password").
- **Non-Functional Requirements:** Define system constraints (e.g., "System must respond within 2 seconds").
- **Business Requirements:** High-level goals of the software (e.g., "Increase customer engagement by 20%").

3. Choosing a Requirements Modeling Technique

There are various techniques to represent requirements:

A. Use Case Modeling (UML Diagram)

- Uses **actors** (users) and **use cases** (functions).
- Shows how users interact with the system.
- **Example:** A "Login" use case shows that a user enters credentials, and the system verifies them.

B. Data Flow Diagrams (DFD)

- Represents the flow of information in the system.
- Shows how data is processed and stored.
- **Example:** A DFD for an ATM system shows how user input is processed by the bank database.

C. Entity-Relationship Diagram (ERD)

- Used in database modeling.
- Defines relationships between entities (e.g., Users, Orders, Payments).

D. State Transition Diagrams

- Represents different states of an object in the system.
- **Example:** A document in a workflow system may have states like "Draft," "Under Review," and "Approved."

4. Validating the Requirements Model

- Ensure completeness, consistency, and feasibility.
- Conduct reviews with stakeholders.
- Identify missing or conflicting requirements.

Example:

- If a requirement states that the system should allow **password recovery**, but another says **no user information should be stored**, there's a conflict.

5. Documenting the Model (Software Requirement Specification - SRS)

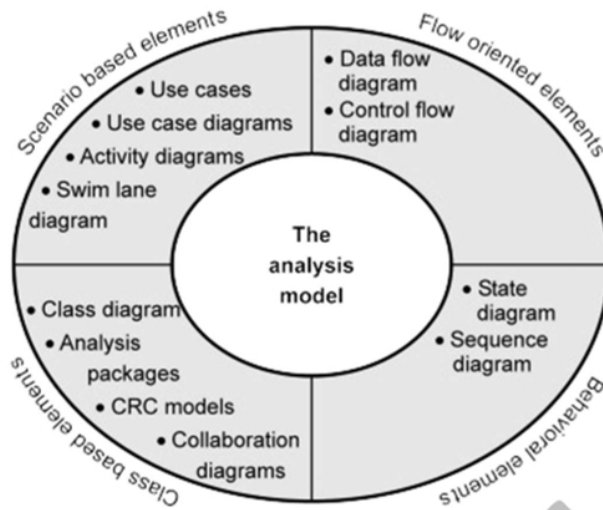
- Clearly defines system requirements for developers, testers, and stakeholders.
- Includes **use cases, diagrams, constraints, and assumptions**.

Example SRS Sections:

1. **Introduction** (Scope, Objectives)
2. **Functional Requirements** (Login, Registration, Checkout)
3. **Non-Functional Requirements** (Performance, Security)
4. **Data Requirements** (ERD, Database Schema)
5. **System Models** (DFD, UML Diagrams)

1.14 Elements of the Requirements Model

A **Requirements Model** helps define, organize, and visualize the system's functional and non-functional aspects. It acts as a blueprint for software development.



1. Scenario-Based Elements (*Describes System Interactions*)

These elements focus on **how users interact** with the system.

- **Use Cases** – Define system functionalities from the user's perspective.
- **User Stories** – Short descriptions of user needs in simple language.
- **Use Case Diagrams** – Graphical representation of actors and use cases.

Example:

- **Actor:** Customer
- **Use Case:** "Place an Order"
- **Scenario:** User selects items, enters details, and confirms payment.

2. Behavioral Elements (*Describe System Reactions & State Changes*)

These define **how the system behaves** under different conditions.

- **State Transition Diagrams** – Show how the system moves between different states (e.g., "Draft" → "Submitted" → "Approved").
- **Sequence Diagrams** – Illustrate interactions between system components.
- **Activity Diagrams** – Represent workflows and actions.

Example:

- A **document approval system** has states: "Pending" → "Reviewed" → "Approved."

3. Class-Based Elements (*Define System Objects & Relationships*)

These represent the **data structure** and system objects.

- **Class Diagrams** – Define system objects, their attributes, and relationships.
- **Entity-Relationship Diagrams (ERD)** – Represent database entities and their relationships.

Example:

- **Entities:** User, Order, Product
- **Relationship:** A User **places** an Order, and an Order **contains** Products.

4. Flow-Oriented Elements (*Define Data Processing & System Logic*)

These elements define **how information is processed** in the system.

- **Data Flow Diagrams (DFD)** – Illustrate data movement between system components.
- **Control Flow Diagrams** – Show the logical flow of execution in a system.

Example:

- In an **ATM system**, a DFD shows the process: *User enters PIN* → *System verifies PIN* → *User selects action (withdraw, balance check, etc.)*.

1.15 Writing a Use Case

1. Definition of Use Cases

A **use case** represents the interaction between an actor (user or external system) and the system to achieve a specific goal. It defines **how the system behaves** in different scenarios.

Example: In a **library system**, the use case "**Reserve a Book**" describes the steps a student follows to search, find, and reserve a book. Use cases can be represented as follows:



2. Components of a Use Case

A. Actors

An **actor** is an entity (person, system, or device) that interacts with the system.

- **Primary Actor** – Initiates the interaction (e.g., Student).
- **Secondary Actor** – Supports the process (e.g., Library Database).

Example:

- **Student** is an actor in a library system.
- **Bank System** can be an actor in an online payment system.

B. Use Cases

Use cases **describe system functionalities** in terms of user interactions.

Example (Library System Use Cases):

1. **Search for a Book**
2. **Reserve a Book**
3. **Borrow a Book**
4. **Return a Book**
5. **Pay Fine**

Each use case represents a **specific behavior** of the system.

C. Relationships in Use Cases

1. Association: Defines interaction between an actor and a use case.

Example: A **student** is associated with the "**Reserve a Book**" use case.

2. Include Relationship: Represents a **mandatory** use case that must be executed as part of another use case.

Example: The "**Login**" use case is **included** in "**Reserve a Book**" because a student must log in before reserving a book.

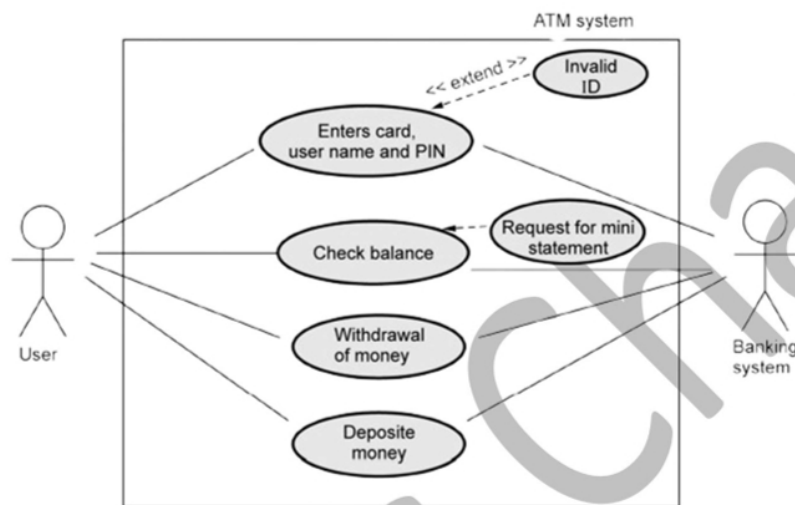
3. Extend Relationship: Represents a **conditional** use case that **extends** another use case when needed.

Example: A "**Late Fee Calculation**" use case **extends** the "**Return Book**" use case only if the book is returned late.

4. Generalization: Defines an **inheritance relationship** between use cases or actors.

Example: A "**Librarian**" is a specialized actor of "**Library Staff**" because they have additional privileges.

Q. Design use case diagram for user interaction with ATM system.



For an ATM system following are the scenarios with the user.

1. User enters the card, types user name and PIN.
2. User makes enquiry for balance. He may demand for getting the mini statement.
3. User withdraws the desired amount of money.
4. User may deposit some money.
5. Finally, user closes the session.

1.16 Negotiating Requirements

Requirement negotiation is the process of resolving conflicts, inconsistencies, or differences among stakeholders regarding software requirements. It ensures that all parties agree on a common set of requirements before development begins.

Example:

- A customer wants a highly secure system, but a developer is concerned about performance trade-offs.
- The negotiation involves balancing security and performance to find optimal solution.

Steps in Requirement Negotiation

1. Identify Conflicts

- Review requirement documents and feedback.
- Find conflicting needs (e.g., cost vs. features, usability vs. complexity).

2. Analyze Stakeholder Interests

- Understand each stakeholder’s concerns.
- Prioritize based on project constraints (budget, timeline, feasibility).

3. Discuss and Resolve Differences

- Conduct meetings or workshops to discuss conflicting requirements.
- Use trade-offs or compromises to find a middle ground.

4. Document Agreed Requirements

- Clearly define final requirements and ensure all parties agree.
- Avoid vague descriptions that could lead to future disputes.

5. Validate with Stakeholders

- Get formal approval before moving forward with development.

Example of Requirement Negotiation

Scenario: Developing an E-Commerce Website

Stakeholder	Requirement	Conflict	Negotiation Solution
Customer	High-resolution images for products	Increases page load time	Optimize image compression
Developer	Fast-loading website	Requires lower-quality images	Use CDN and lazy loading
Marketing Team	More ads for revenue	Affects user experience	Limit ad frequency

1.17 Validating Requirements

Requirement validation is the process of ensuring that the gathered software requirements are **complete, consistent, unambiguous, and feasible** before development begins. The goal is to confirm that the requirements meet the needs of stakeholders and align with business objectives.

Without proper validation, errors in requirements can lead to costly rework, project delays, or software that fails to meet user expectations.

Steps in Requirement Validation

1. Review Requirements

- Check for completeness, correctness, and clarity.
- Identify missing or conflicting requirements.

2. Perform Requirement Analysis

- Ensure requirements are realistic and feasible.
- Evaluate if they align with technical and business constraints.

3. Validate with Stakeholders

- Conduct meetings with users, customers, and developers to confirm requirements.
- Gather feedback and make necessary adjustments.

4. Verify Consistency and Traceability

- Ensure there are no contradictions between different requirements.
- Check that each requirement is traceable to business objectives.

5. Perform Requirement Testing

- Create test cases based on requirements to check if they can be implemented correctly.

Example of Requirement Validation

Scenario: Online Banking System

Requirement	Validation Check	Findings	Solution
The system must allow fund transfers	Can the feature be implemented securely?	Security concerns in data transmission	Implement encryption protocols
The login should take no more than 2 seconds	Is this performance goal realistic?	Server response time too slow	Optimize database queries
The system should support 1 million users	Does infrastructure support this?	Scalability concerns	Use cloud-based hosting

1.17 Quality Function Deployment (QFD)

Quality Function Deployment (QFD) is a structured approach used to **translate customer needs into specific engineering requirements**. It ensures that the final software product meets user expectations by systematically prioritizing and aligning software features with business goals.

QFD is often represented using a **House of Quality (HoQ)** matrix, which maps customer requirements to technical requirements.

Key Objectives of QFD

- Identify **customer needs** (also called "Voice of the Customer").
- Convert customer needs into **measurable technical requirements**.
- Prioritize requirements based on **business and technical feasibility**.
- Minimize errors by ensuring **alignment between customer expectations and software design**.

Types of Requirements in QFD

QFD classifies requirements into **four types**:

1. Normal Requirements (Expected Needs)

- These are the **basic features** that users expect in the software.
- If these features are missing, users will be dissatisfied, but having them does not necessarily increase satisfaction.

Example: A banking app must allow users to **check account balance** and **transfer funds**.

2. Performance Requirements (Variable Needs)

- These requirements directly impact user satisfaction.
- The better the software performs these functions; the more satisfied users will be.

Example: A faster response time in a search engine improves user experience.

3. Exciting Requirements (Delighters)

- Features that users **did not explicitly request** but, when provided, greatly enhance satisfaction.
- Their absence does not cause dissatisfaction, but their presence creates a competitive advantage.

Example: A banking app automatically categorizing expenses and providing financial insights.

4. Unspoken Requirements (Latent Needs)

- These are needs that customers may not be aware of but still expect when using the system.
- Identifying these requires deep analysis and experience.

Example: Auto-save functionality in word-processing software prevents data loss, even though users may not explicitly request it.

Steps in QFD Process

1. **Gather Customer Requirements:** Conduct surveys, interviews, and market research.
2. **Prioritize Requirements:** Use techniques like ranking or weighting based on importance.
3. **Translate into Technical Specifications:** Define measurable engineering or software design requirements.
4. **Develop the House of Quality (HoQ) Matrix:** A visual representation of how customer needs relate to technical requirements.
5. **Refine and Implement the Requirements:** Develop prototypes, validate with users, and adjust based on feedback.

Example of QFD in Software Development

For an e-commerce website, different types of requirements can be:

Requirement Type	Example	Impact
Normal Requirement	Secure login with two-factor authentication	Essential, expected by users
Performance Requirement	Website should load in under 2 seconds	Directly impacts user satisfaction
Exciting Requirement	AI-based product recommendations	Increases customer engagement
Unspoken Requirement	Auto-save cart items across devices	Prevents frustration, improves usability

1.17 Data Flow Diagram (DFD)

A **Data Flow Diagram (DFD)** is a graphical representation of how data moves through a system. It illustrates:

- **Processes** that transform data
- **Data stores** where data is held
- **External entities** (sources and destinations of data)
- **Data flows** (movement of data between elements)

DFDs are used to **analyze, model, and document** information flow within a system, making them essential in requirement analysis and system design.

Levels in Data Flow Diagram (DFD)

DFDs can be divided into different levels, which provide varying degrees of detail about the system. The following are the four levels of DFDs:

1. **Level 0 DFD**
2. **Level 1 DFD**
3. **Level 2 DFD**
4. **Level 3 DFD**

The choice of DFD level depends on the complexity of the system and the level of detail required to understand the system. Higher levels of DFD provide a broad overview of the system, while lower levels provide more detail about the system's processes, data flows, and data stores. A combination of different levels of DFD can provide a complete understanding of the system.

Level 0 Data Flow Diagram (DFD)

Level 0 is the highest-level Data Flow Diagram (DFD), which provides an overview of the entire system. It shows the major processes, data flows, and data stores in the system, without providing any details about the internal workings of these processes.

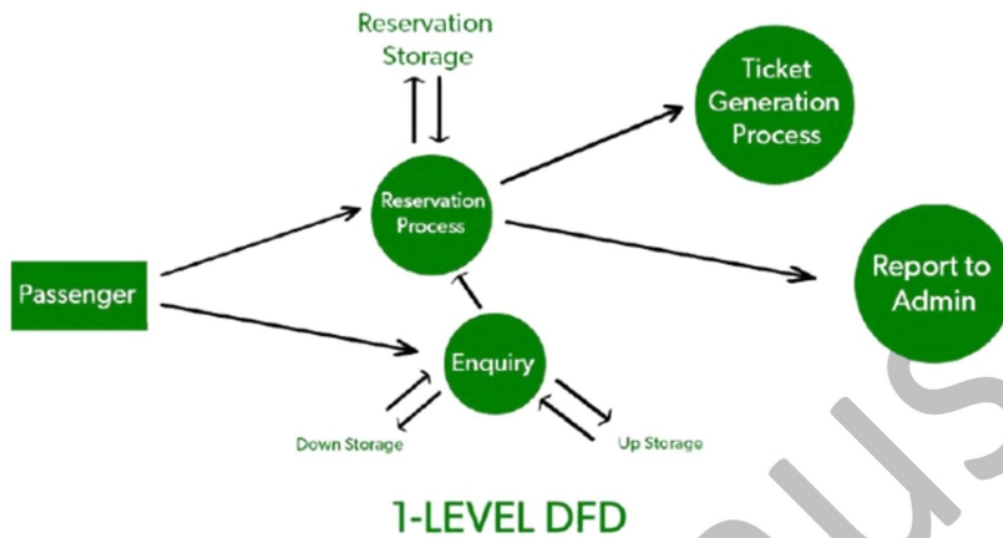
It is also known as a context diagram. It's designed to be an abstraction view, showing the system as a single process with its relationship to external entities. It represents the entire system as a single bubble with input and output data indicated by incoming/outgoing arrows.



1-Level Data Flow Diagram (DFD)

1-Level provides a more detailed view of the system by breaking down the major processes identified in the level 0 Data Flow Diagram (DFD) into sub-processes. Each sub-process is depicted as a separate process on the level 1 Data Flow Diagram (DFD). The data flows and data stores associated with each sub-process are also shown.

In 1-level Data Flow Diagram (DFD), the context diagram is decomposed into multiple bubbles/processes. In this level, we highlight the main functions of the system and breakdown the high-level process of 0-level Data Flow Diagram (DFD) into subprocesses.



2-Level Data Flow Diagram (DFD)

2-Level provides an even more detailed view of the system by breaking down the sub-processes identified in the level 1 Data Flow Diagram (DFD) into further sub-processes. Each sub-process is depicted as a separate process on the level 2 DFD. The data flows and data stores associated with each sub-process are also shown.

2-Level Data Flow Diagram (DFD) goes one step deeper into parts of 1-level DFD. It can be used to plan or record the specific/necessary detail about the system's functioning.

